

Mooncake: 以 KVCache 为中心的云上 LLM 推理软件栈

KVCache 软件生态和部署
马腾 - 阿里云飞天实验室



Content 目录

01 KVCache 解耦架构

02 Mooncake 项目

03 相关进展

04 云上 AI Stack



Part 01

KVCache 解耦架构



KVCache 共享和复用基本原理

- 大模型辅助读论文场景为例



所有查询
可复用 {

热点
论文
复用
率高 {

System Prompt: 你是一个论文阅读助手, ...

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez*[†]
University of Toronto
aidan@cs.toronto.edu

Lukasz Kaiser*
Google Brain
lukaszkaizer@google.com

Illia Polosukhin*[†]
illia.polosukhin@gmail.com

Mooncake: A KVCache-centric Disaggregated
Architecture for LLM Serving

Ruoyu Qin^{♥1} Zheming Li^{♥1} Weiran He[♥]
Mingxing Zhang^{♥2} Yongwei Wu[♥] Weimin Zheng[♥] Xinran Xu^{♥2}
[♥]Moonshot AI [♥]Tsinghua University

User Question 1:
总结一下这篇论文

User Question 2:
这篇论文关键创新是什么

User Question 3:
有哪些相关研究?

User Question 4:
可以进一步探索什么?

可复用
长度长

问题短



不同解耦架构带来的优势



将 Prefill 与 Decode 解耦，
让“算力 - 吃紧”与“带宽 - 吃紧”任务各得其所，推理延迟 - 吞吐双降

PD 分离



把 Attention 计算和
Expert 路由 / 计算分到异构硬件，CPU 缓存 KV、GPU 稀疏激活专家，MoE 模型显存换带宽，成本直降

AF 分离



把多模态编码、KV 生成、自回归解码拆成三阶段流水线，单阶段内存爆降 95%，批尺寸 $\times 20$ ，长图长视频秒级 TTFT。

EPD 分离

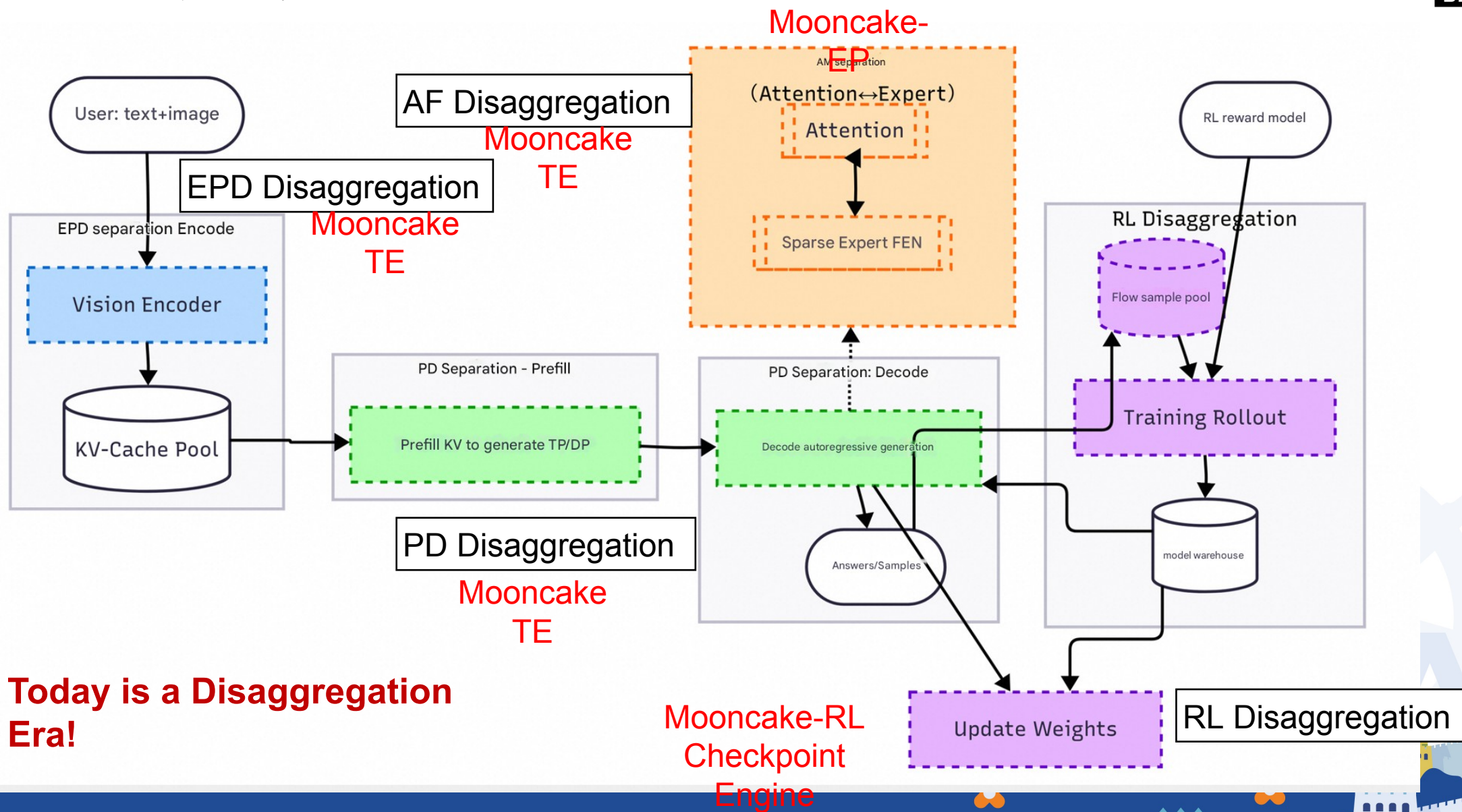


把 RLHF 中的在线样本生成（推理）与策略梯度更新（训练）彻底分开，独立扩缩容、异构部署，训练吞吐提升 2.6 $\times$ ，成本再降 30%。

推训分离



不同解耦架构带来的优势

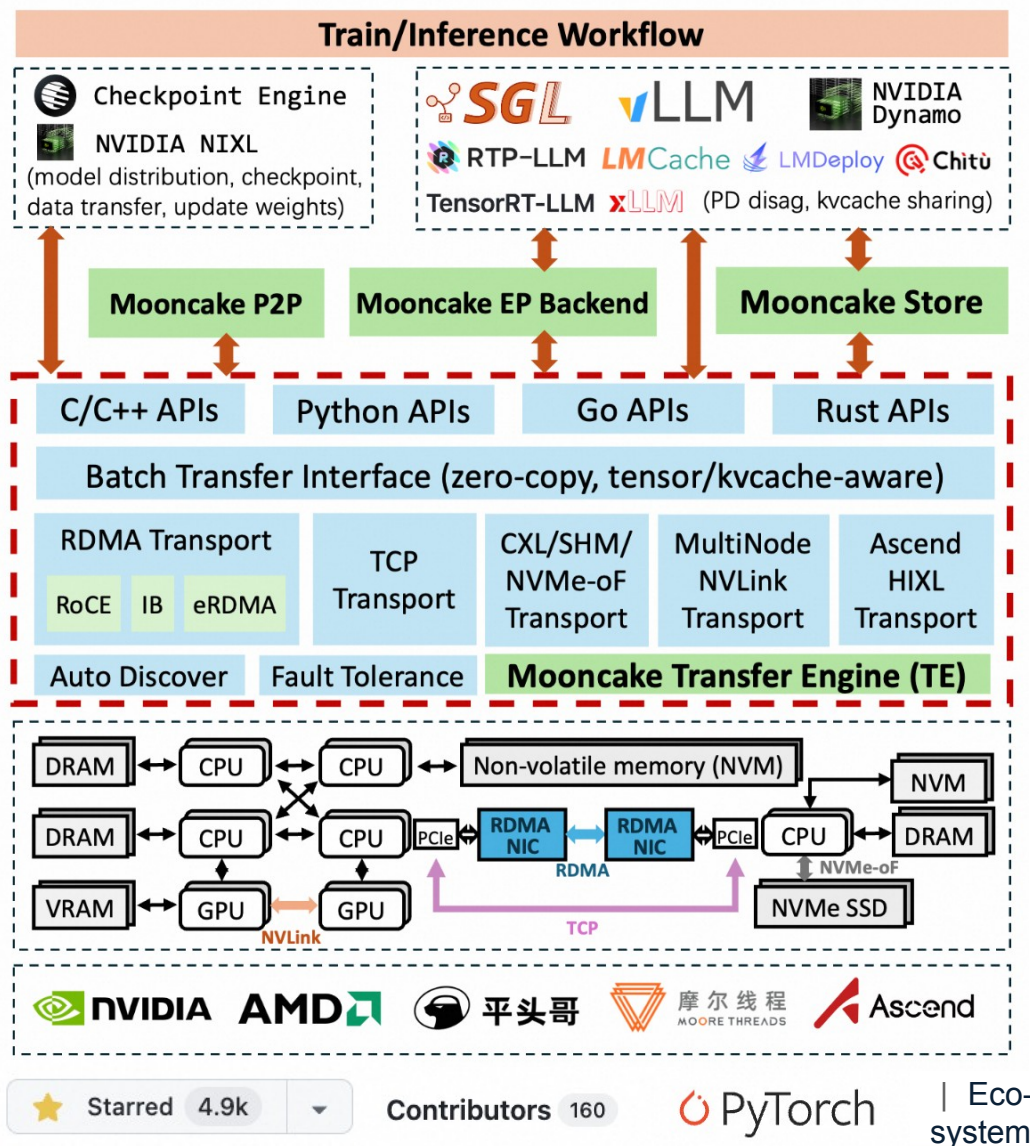


Part 02

Mooncake 项目



Mooncake 项目架构



Transfer Engine :

- 全链路零拷贝、多网卡池化
- 最高 8*400Gbps 聚合带宽、拓扑感知
- 支持动态拓扑，故障容错，负载均衡，多协议
- 更充分地发挥高性能网卡的优势
- 相比 nccl 更加灵活

KVCache Store :

- 充分利用 GPU 集群中闲置的内存和带宽
- 省成本的同时降低响应延迟
- 透明多级缓存，VRAM/DRAM/SSD/Remote
- 进一步下沉到底层廉价存储

Elastic EP :

- 基于点对点通信的 EP 策略和 Torch 后端
- 支持弹性和容错，易于部署

Production Ready:

- AC2 镜像 + K8S 部署
- 提供 whl 包一键安装
- Nightly Build & End-to-end CI Test

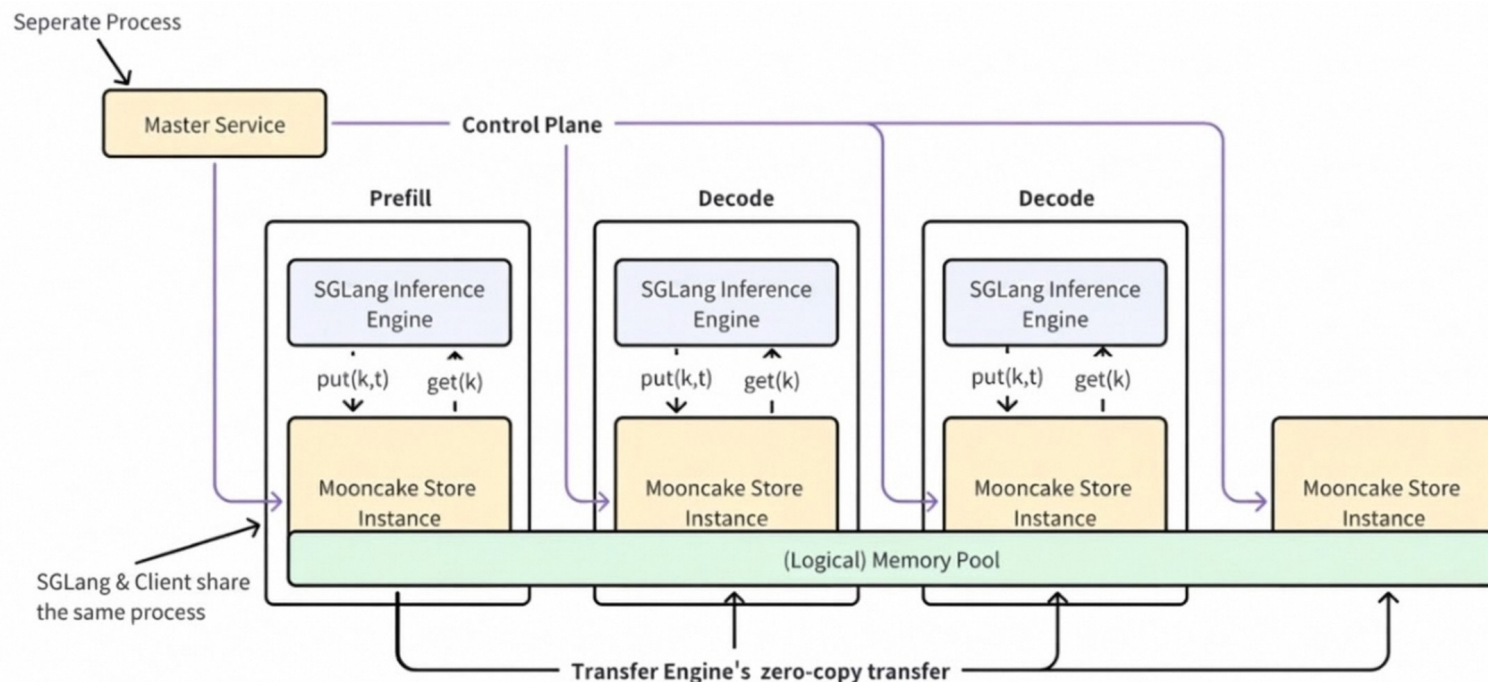
Mooncake Store

与传统缓存系统的区别：

- Key 由 value 通过哈希计算得到，无需 update 操作
- 支持 Lease，没有版本管理的需求

灵活性与可定制性：

- 提供底层对象存储和管理功能。
- 具体缓存策略由上层框架 / 用户实现（如 SGLang/



系统特性：

- 提供 KV 复用率，TTFT 等指标
- etcd 可靠性服务，高可用模式
- 数据持久化能力
- 提供分层存储能力（NFS/3FS）
- 提供 Restful API/ 独立服务
- 多语言接口 / 零拷贝能力



Part 03

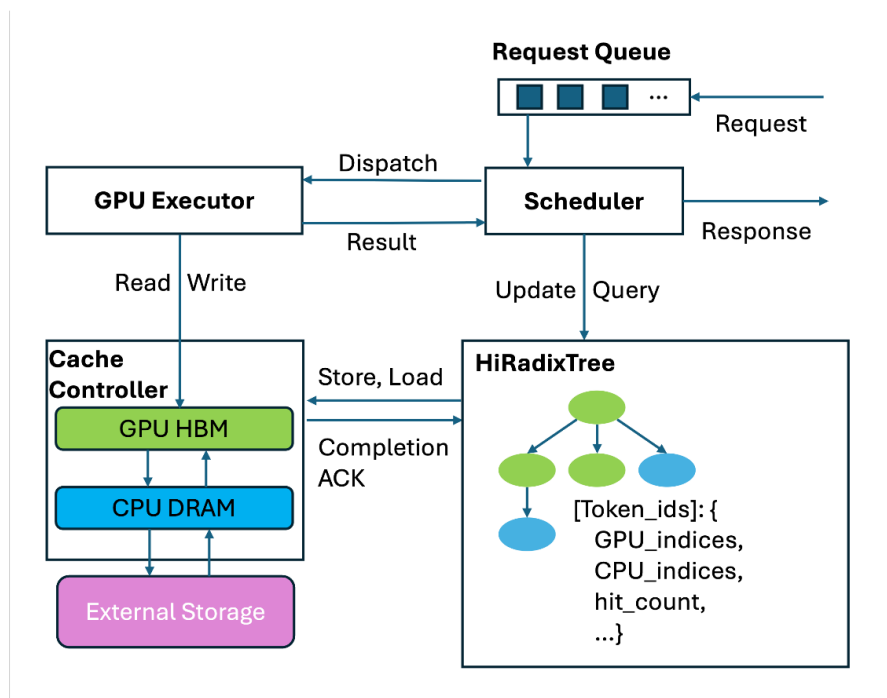
相关进展



Mooncake 对于大模型推理生态支持

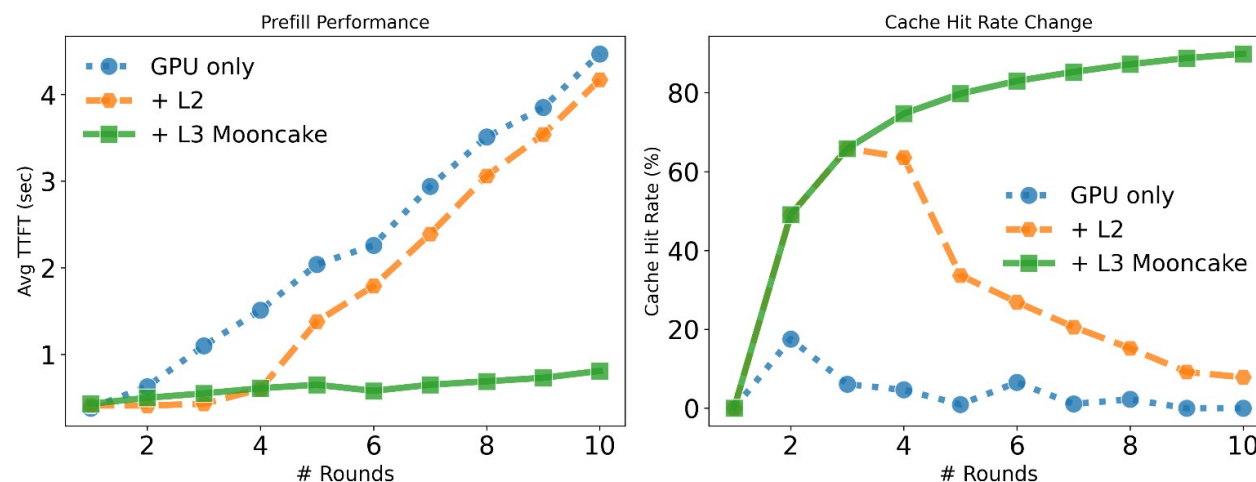
Feature	Project	Type	Transfer	EP/Torch Backend	KVCache Store	Ckpt Engine
Inference	SGLang (Omni)	Inference	✓	✓	✓	✓
	vLLM V0	Inference	✓	✗	✓	✓
	vLLM V1 (Omni)	Inference	✓	WIP	✓ (w LMCache/Nixl)	✗
	LMDeploy	Inference	✓	✗	✓	✗
	Chitu	Inference	✓	✗	✗	✗
	RTP (Alibaba)	Inference	✗	✗	✓	✗
Middleware	KVCM (Alibaba)	Middleware	✓	-	✓	-
	TBase (Ant)	Middleware	✓	-	✗	-
	Dynamo	Framework	✓ (w/ Nixl)	✗	✗	✗
	LMCache	Middleware	✗	-	✓	-
RL Post-Training	Slime	RL	WIP	-	✓	-
	ROLL	RL	✗	-	✓	-

Mooncake Store + HiCache (KVCache Offloading)



- 流水线掩盖：将 GPU 计算与 KV 数据传输（H2D/D2H）重叠执行，智能预取隐藏 I/O 延迟
- 零拷贝：利用 Mooncake 的 RDMA 机制与 GPU Direct 技术，绕过 **CPU** 拷贝，降低通信开销

SGLang HiCache with Mooncake Backend on Multi-turn Conversation Benchmark



引入 Mooncake 作为 L3，实现跨机 **KV** 共享与持久化，支持无限长上下文（Infinite Context），解决多实例间的重复计算问题

HiCache 高性能 IO Kernel



Part 04

云上 AI Stack



当前挑战：显存墙与架构演进

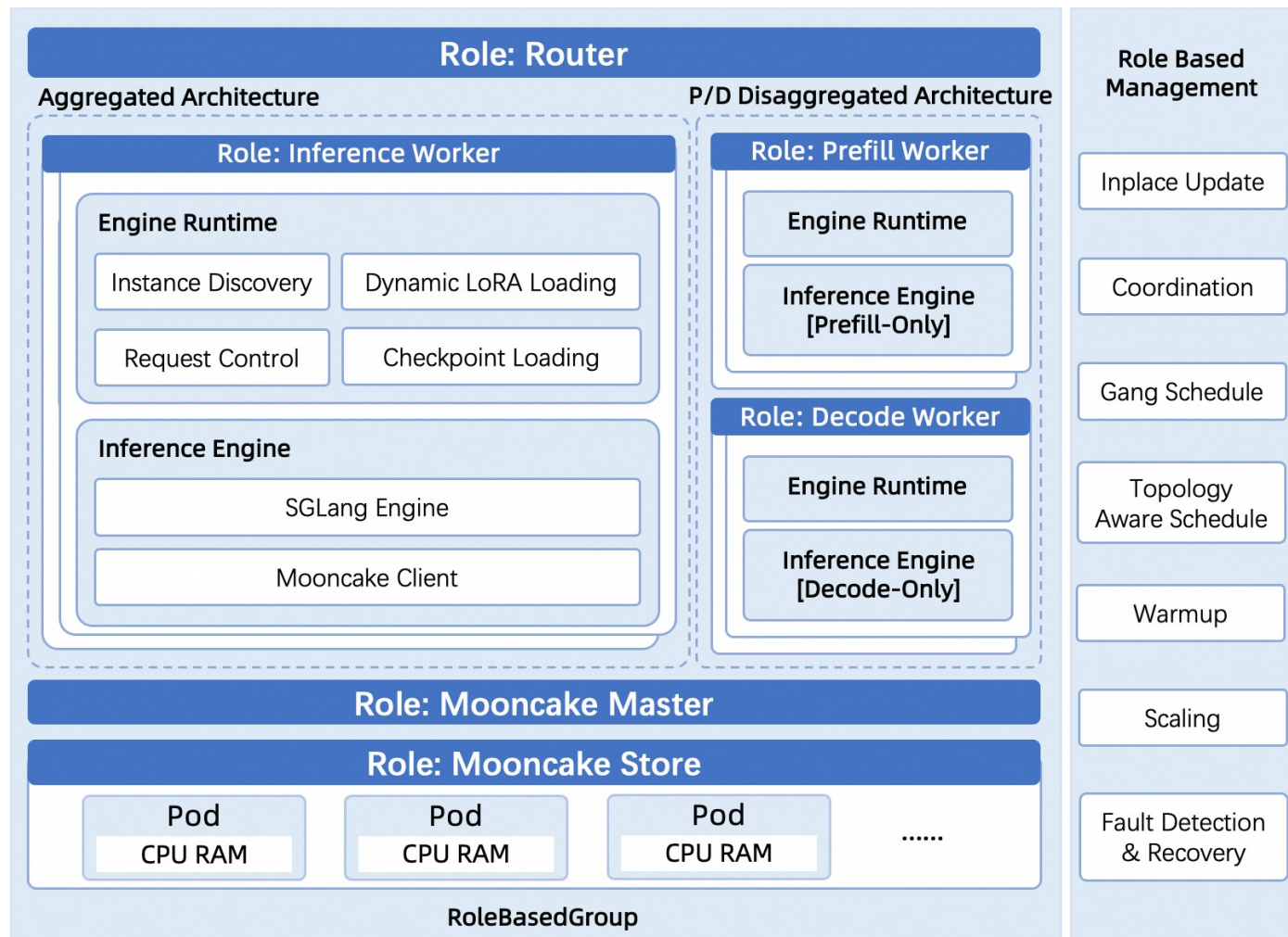
- **显存压力：** 在长上下文和高并发场景下，KVCache 显存占用常超 70%，单机 GPU HBM 和 CPU DRAM 已难以为继。
- **架构演进：** 推理架构正从单体向分布式演进（Prefill-Decode 分离、KVCache 外置），以实现跨请求缓存共享与弹性伸缩。
- **运维痛点：** 传统的 K8s Workload 难以处理推理角色间的强依赖与拓扑感知，且滚动升级常导致缓存丢失与性能抖动。

核心解决方案： **RBG + Mooncake**

- **Mooncake（存储引擎）：** 分布式 KVCache 存储，提供高吞吐、低延迟的 L3 缓存服务。
- **RBG（编排引擎）：** RoleBasedGroup 是面向 AI 推理的 K8s 原生 API，统一管理多角色协同、部署与弹性。
- **价值主张：** 将 Mooncake 作为 RBG 编排下的补充角色，实现性能最大化（L3 缓存）与生产级稳定性（原地无感升级）。



部署架构: Router+RBG+SGLang+Mooncake Store-LLM



系统角色构成

- **SGLang Router** : 统一入口与流量调度, 智能分发至 Prefill 或 Decode 后端
- **Prefill Worker** : 计算密集型, 负责 Prompt 前向计算并生成初始 KVCache
- **Decode Worker** : 延迟敏感型, 依赖 KVCache 进行 Token 逐个解码
- **Mooncake Store/Master** : 独立外置存储角色, 持久化存储 KVCache

协同工作流

- 所有角色通过 RBG 进行紧密集成与协同
- 利用 RDMA 网络在计算节点与 Mooncake 存储节点间高速传输 KVCache
- Router 根据负载动态调度, 实现动态批处理与连续批处理



解决行业难题——原地无感升级 (In-place Update)



痛点：传统的滚动升级风险

- Mooncake 作为有状态服务，传统 Pod 重建会导致内存中 KVCache 丢失
- 后果：活跃会话中断，被迫重新 Prefill，导致 P99 延迟毛刺飙升，吞吐量断崖式下跌

解决方案：RBG 原地升级 + 持久化

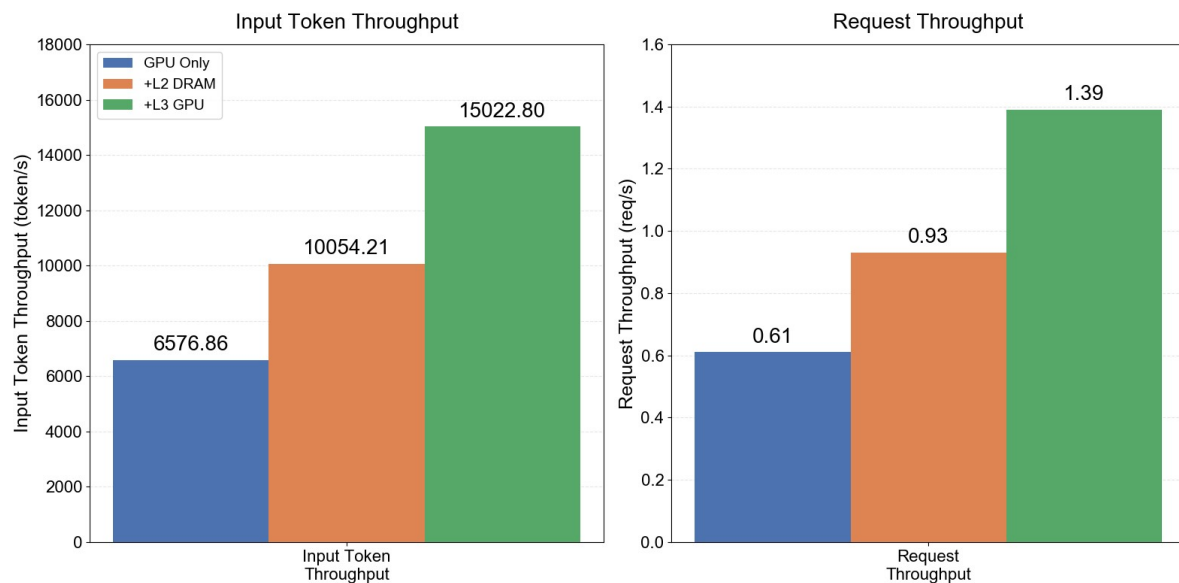
- **Mooncake** 本地持久化：支持 KVCache 快照至共享内存 / 本地盘，进程重启后快速恢复数据
- **RBG** 原地升级策略：K8s 原生 API 实现原地替换容器镜像，复用节点资源，避免 Pod 重建

最终效果

- 升级无感，服务不抖：彻底避免缓存失效导致的重计算，保障端到端稳定性
- 运维自动化：将复杂的分布式协同升级转化为标准化的运维能力

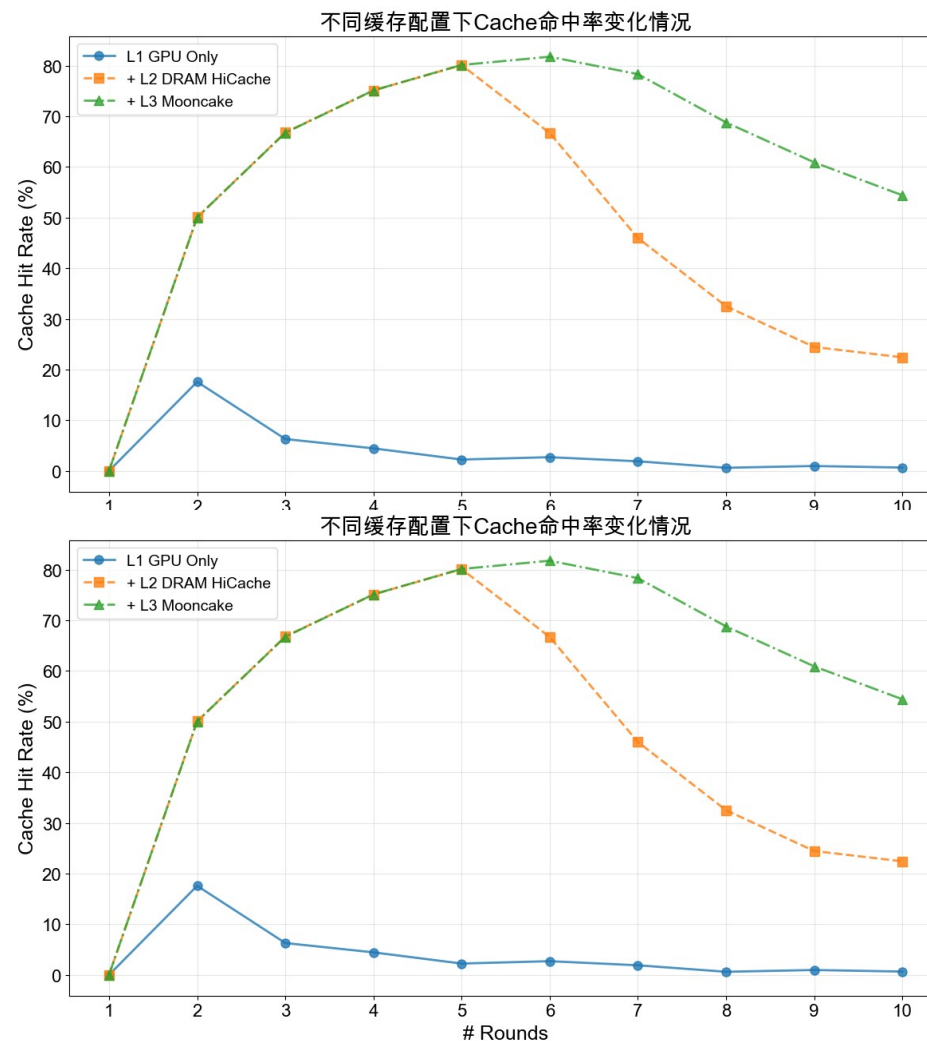


ACK 场景下部署效果

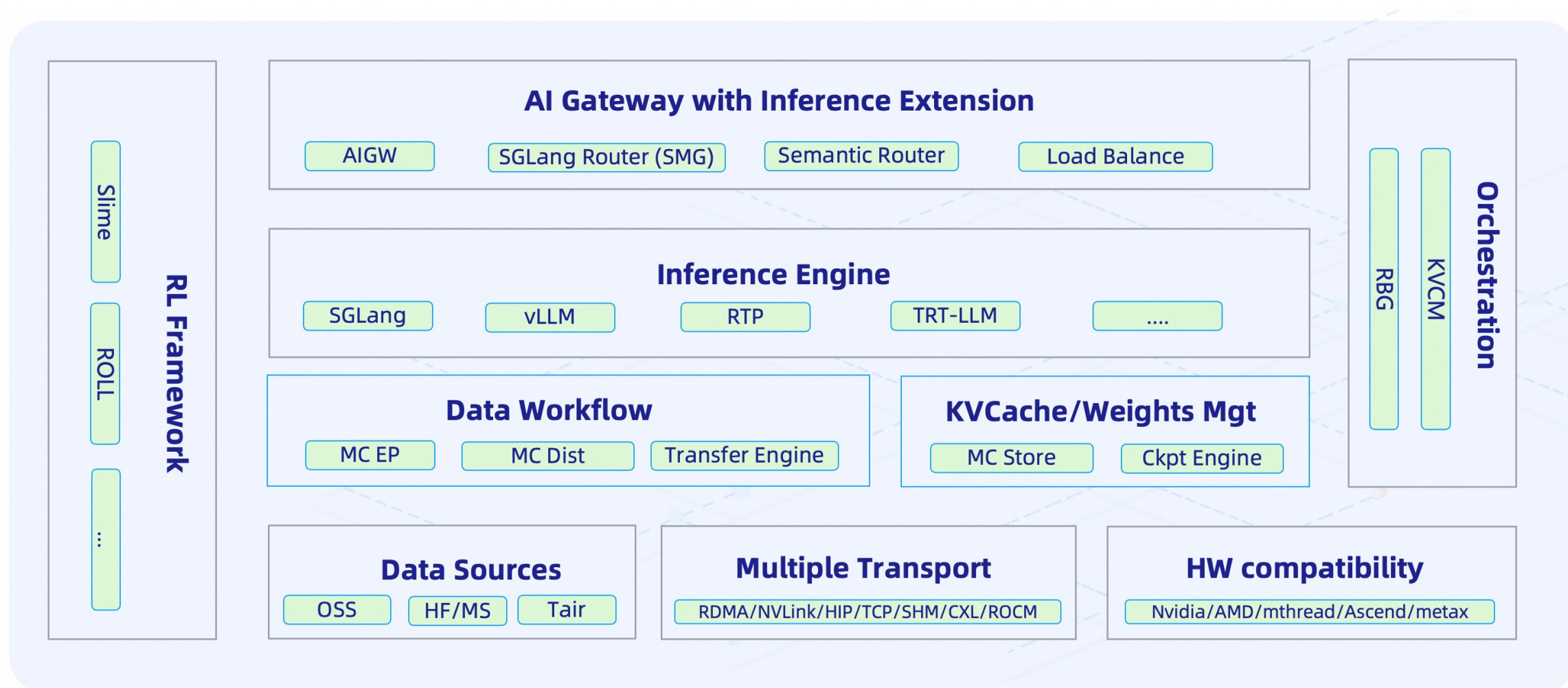


结论:

引入 L3 Mooncake 缓存后，能够显著提升缓存命中率，大幅降低延迟并提升系统整体吞吐量，实现性能与成本的平衡



Mooncake AI Production Stack



Thanks.

